

Field Training Management System

Course: Field Training

Project Report (Phases 1–5)

Team Members:

- Yassin Ahmed Mohammed Kamaleldeen Nasra (23101503)
- Mahmoud Mohammed Fathy (23101475)
- Youssef Nader Attia (23101519)
- Youssef Adel Boshra (23101513)
- Yassin Ali Attia (23101505)

Table of Contents

[TOC will be generated upon export in Word]

Phase 1 – Business Rules and Requirements

Application Description

The Field Training Management System aims to streamline the enrollment and monitoring process for 'Field Training 2' at the university. This system ensures effective communication between students, faculty, and organizations. It also facilitates mentor assignment, internship approvals, and report tracking, all within a centralized database.

Business Rules

For Students:

1. Field Training 2 is mandatory (2 credit hours), starting from academic level 3.
2. Only students at level 3 or above can register.
3. Report submissions and employer feedback are mandatory.
4. Each student is assigned an internal and an external mentor.
5. Applications are only allowed for coordinator-approved organizations.

For Evaluators:

1. Evaluate internship reports and assign grades.
2. Provide consistent weekly updates via the system.

For Organizations:

1. Must be coordinator-approved before appearing in the system.

For External Mentors:

1. Must evaluate assigned students.
2. Evaluations contribute to the final grade.

For Admins:

1. Can manage all user roles and internship data.
2. Must ensure weekly and final reports are uploaded.
3. The system must support efficient queries and data integrity.

Phase 2 – Requirements and Queries

Functional Requirements

1. Enable student internship registration.
2. Allow coordinators to approve/reject organizations.
3. Track internship application status.
4. Generate weekly student evaluations and internship reports.
5. Produce end-of-semester reports.
6. Admin dashboard for full data management and integrity enforcement.
7. Enable simple and fast data querying.

Sample Queries

Student Level Queries:

- Get students eligible for internships.
- List internships a student has applied for.
- View student submissions and evaluations.
- Identify internal and external mentors assigned.

Organization/Internship Queries:

- List all approved organizations.
- Show pending organization approval requests.
- Retrieve details of a specific internship.
- Count students at a specific organization.

Mentor & Evaluation Queries:

- View students assigned to a mentor.
- Access evaluations by external mentors.
- Generate weekly reports per student.

Admin & Reporting Queries:

- Generate weekly and semester-end reports.
- Retrieve top-performing students.
- List all students, mentors, and organizations.
- View past internships for comparative analysis.

Phase 3 – Data Modeling and Normalization

Entities and Attributes

Summarized EERD entities include:

- User, Student, Admin, Mentor, Internal Mentor, External Mentor
- Internship Coordinator, Internship Evaluator, Internship, Organization
- Internship Report, Evaluation, Apply_For, Host, Phones, UserPhoneNumbers, Submits

Relationships

- Manages: Admin → Users (1:M)
- Apply_For: Student ↔ Internship (M:M)
- Approves: Coordinator → Internship (1:M)
- Hosts: Organization ↔ Internship (M:M)
- Guides: Internal Mentor → Student (1:M)
- Evaluates: External Mentor / Evaluator → Internship (1:M)
- Submits: Student ↔ Internship_Report (M:M)
- Submits_Evaluation: Evaluator / External Mentor → Evaluation (M:1)

Normalization Summary

All relations are normalized to Third Normal Form (3NF):

- 1NF: No multivalued attributes
- 2NF: No partial dependencies
- 3NF: No transitive dependencies

Examples:

- StudentID → Academic_Level, Field_Of_Study, etc. – Fully dependent
- Composite PKs in Apply_For, Submits, and Host are clearly defined with full dependency.

Phase 4 – SQL DDL Script Summary

Database Name: FIELD_TRAINING_MANAGEMENT

Team: The Primary Keys

User Management Tables(User_UserPhoneNumbers_Admin)

This section contains tables for managing users and their associated phone numbers, along with a special table for Admin users.

```
CREATE TABLE User (
    UserID INT PRIMARY KEY,
    Name VARCHAR(100),
    Email VARCHAR(100) UNIQUE,
    Role VARCHAR(50)
);

CREATE TABLE UserPhoneNumbers (
    PhoneNumber VARCHAR(15),
    UserID INT,
    PRIMARY KEY (PhoneNumber, UserID),
    FOREIGN KEY (UserID) REFERENCES User(UserID)
);

CREATE TABLE Admin (
    AdminID INT PRIMARY KEY,
    UserID INT UNIQUE,
    FOREIGN KEY (UserID) REFERENCES User(UserID)
);
```

Mentor & Evaluator

Structure(Mentor_Evaluator_InternalMentor_ExternalMentor)

This section defines tables related to mentors and evaluators. The structure accommodates both internal and external mentors along with internship evaluators.

```
CREATE TABLE Mentor (
    MentorID INT PRIMARY KEY,
    Experience_Years INT,
    Students_Assigned_Number INT,
    FOREIGN KEY (MentorID) REFERENCES User(UserID)
);

CREATE TABLE Internship_Evaluator (
    Internship_EvaluatorID INT PRIMARY KEY,
```

```

        Role VARCHAR(100),
        FOREIGN KEY (Internship_EvaluatorID) REFERENCES User(UserID)
    );

CREATE TABLE InternalMentor (
    Internal_Mentor_ID INT PRIMARY KEY,
    Department VARCHAR(100),
    FOREIGN KEY (Internal_Mentor_ID) REFERENCES Mentor(MentorID)
);

CREATE TABLE ExternalMentor (
    External_Mentor_ID INT PRIMARY KEY,
    Association_Name VARCHAR(100),
    FOREIGN KEY (External_Mentor_ID) REFERENCES Mentor(MentorID)
);

```

Student & Reports(InternshipReport_Student_Submits)

This section contains tables for managing students, their internship reports, and report submissions.

```

CREATE TABLE Internship_Report (
    Report_ID INT PRIMARY KEY,
    Submission_Date DATE,
    Report_Grade DECIMAL(5,2),
    Internship_EvaluatorID INT,
    FOREIGN KEY (Internship_EvaluatorID) REFERENCES
    Internship_Evaluator(Internship_EvaluatorID)
);

CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    Academic_Level INT,
    Field_Of_Study VARCHAR(100),
    Internship_Status VARCHAR(50),
    Internal_Mentor_ID INT,
    ReportID INT,
    FOREIGN KEY (StudentID) REFERENCES User(UserID),
    FOREIGN KEY (Internal_Mentor_ID) REFERENCES
    InternalMentor(Internal_Mentor_ID),
    FOREIGN KEY (ReportID) REFERENCES Internship_Report(Report_ID)
);

CREATE TABLE Submits (
    StudentID INT,
    ReportID INT,
    PRIMARY KEY (StudentID, ReportID),
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),

```

```

        FOREIGN KEY (ReportID) REFERENCES Internship_Report (Report_ID)
    );

```

Organization & Coordination(InternshipCoordinator_Organization_Phones)

This section deals with internship coordinators, organizations, and their associated phone numbers.

```

CREATE TABLE Internship_Coordinator (
    CoordinatorID INT PRIMARY KEY,
    Internship_Evaluator_ID INT,
    External_Mentor_ID INT,
    FOREIGN KEY (CoordinatorID) REFERENCES User (UserID),
    FOREIGN KEY (Internship_Evaluator_ID) REFERENCES
Internship_Evaluator (Internship_EvaluatorID),
    FOREIGN KEY (External_Mentor_ID) REFERENCES
ExternalMentor (External_Mentor_ID)
);

```

```

CREATE TABLE Organization (
    Organization_ID INT PRIMARY KEY,
    Name VARCHAR(100),
    Email VARCHAR(100) UNIQUE,
    Approval_Status VARCHAR(50),
    CoordinatorID INT,
    FOREIGN KEY (CoordinatorID) REFERENCES
Internship_Coordinator (CoordinatorID)
);

```

```

CREATE TABLE Phones (
    Phone VARCHAR(15),
    Organization_ID INT,
    PRIMARY KEY (Phone, Organization_ID),
    FOREIGN KEY (Organization_ID) REFERENCES
Organization (Organization_ID)
);

```

Internship Tracking(Internship_Host_ApplyFor)

This section tracks internship statuses and the relationship between students, organizations, and internships.

```

CREATE TABLE Internship (
    Internship_ID INT PRIMARY KEY,
    Status VARCHAR(50),
    OrganizationName VARCHAR(100),
    StartDate DATE,

```

```

        EndDate DATE,
        Coordinator_ID INT,
        FOREIGN KEY (Coordinator_ID) REFERENCES
        Internship_Coordinator(CoordinatorID)
    );

CREATE TABLE Host (
    Organization_ID INT,
    Internship_ID INT,
    PRIMARY KEY (Organization_ID, Internship_ID),
    FOREIGN KEY (Organization_ID) REFERENCES
    Organization(Organization_ID),
    FOREIGN KEY (Internship_ID) REFERENCES Internship(Internship_ID)
);

CREATE TABLE Apply_For (
    Internship_ID INT,
    Student_ID INT,
    Application_Status VARCHAR(50),
    PRIMARY KEY (Internship_ID, Student_ID),
    FOREIGN KEY (Internship_ID) REFERENCES Internship(Internship_ID),
    FOREIGN KEY (Student_ID) REFERENCES Student(StudentID)
);

```

Evaluation(Evaluation)

This section contains the table for tracking internship evaluations by internal and external mentors.

```

CREATE TABLE Evaluation (
    Evaluation_ID INT PRIMARY KEY,
    Internal_Mentor_Evaluation DECIMAL(5,2),
    External_Mentor_Evaluation DECIMAL(5,2),
    Grade VARCHAR(5),
    External_Mentor_ID INT,
    Internship_Evaluator_ID INT,
    FOREIGN KEY (External_Mentor_ID) REFERENCES
    ExternalMentor(External_Mentor_ID),
    FOREIGN KEY (Internship_Evaluator_ID) REFERENCES
    Internship_Evaluator(Internship_EvaluatorID)
);

```

Sample Queries

1. Students with Mentors and Internship Status:

```
SELECT
```

```

        S.StudentID,
        U.Name AS StudentName,
        IM.Internal_Mentor_ID,
        MU.Name AS MentorName,
        S.Internship_Status
FROM Student S
JOIN User U ON S.StudentID = U.UserID
LEFT JOIN InternalMentor IM ON S.Internal_Mentor_ID =
IM.Internal_Mentor_ID
LEFT JOIN Mentor M ON IM.Internal_Mentor_ID = M.MentorID
LEFT JOIN User MU ON M.MentorID = MU.UserID;

```

2. Number of Students per Mentor:

```

SELECT
    MU.Name AS MentorName,
    COUNT(S.StudentID) AS StudentCount
FROM InternalMentor IM
JOIN Mentor M ON IM.Internal_Mentor_ID = M.MentorID
JOIN User MU ON M.MentorID = MU.UserID
LEFT JOIN Student S ON IM.Internal_Mentor_ID = S.Internal_Mentor_ID
GROUP BY MU.Name;

```

3. Students and Their Report Grades:

```

SELECT
    U.Name AS StudentName,
    IR.Report_ID,
    IR.Report_Grade,
    IR.Submission_Date
FROM Student S
JOIN User U ON S.StudentID = U.UserID
LEFT JOIN Internship_Report IR ON S.ReportID = IR.Report_ID
ORDER BY IR.Report_Grade DESC;

```

4. Average Report Grades per Evaluator:

```

SELECT
    UE.Name AS EvaluatorName,
    AVG(IR.Report_Grade) AS AverageGrade
FROM Internship_Report IR
JOIN Internship_Evaluator IE ON IR.Internship_EvaluatorID =
IE.Internship_EvaluatorID
JOIN User UE ON IE.Internship_EvaluatorID = UE.UserID
GROUP BY UE.Name;

```


5. Mentors with More Than One Student:

```
SELECT
    MU.Name AS MentorName,
    COUNT(S.StudentID) AS AssignedStudents
FROM InternalMentor IM
JOIN Mentor M ON IM.Internal_Mentor_ID = M.MentorID
JOIN User MU ON M.MentorID = MU.UserID
JOIN Student S ON IM.Internal_Mentor_ID = S.Internal_Mentor_ID
GROUP BY MU.Name
HAVING COUNT(S.StudentID) > 1;
```

6. Users and Their Phone Numbers:

```
SELECT
    U.UserID,
    U.Name,
    U.Role,
    UP.PhoneNumber
FROM User U
LEFT JOIN UserPhoneNumbers UP ON U.UserID = UP.UserID
ORDER BY U.Role, U.Name;
```

7. Approved Organizations:

```
SELECT
    O.Organization_ID,
    O.Name AS OrganizationName,
    O.Industry_Field,
    O.Head_Quarters,
    U.Name AS CoordinatorName
FROM Organization O
JOIN Internship_Coordinator IC ON O.CoordinatorID = IC.CoordinatorID
JOIN User U ON IC.CoordinatorID = U.UserID
WHERE O.Approval_Status = 'Approved';
```

8. Specific Student's Report Grades:

```
sql
CopyEdit
SELECT
    IR.Report_ID,
    IR.Submission_Date,
    IR.Report_Grade
FROM Submits S
JOIN Internship_Report IR ON S.ReportID = IR.Report_ID
WHERE S.StudentID = 2
```

```
ORDER BY IR.Submission_Date;
```

9. Active Internships:

```
SELECT *  
FROM Internship  
WHERE Status = 'Active';
```

10. Students in Academic Level 4:

```
SELECT *  
FROM Student  
WHERE Academic_Level = 4;
```



Coordinator_MentorStudents:

```
sql  
CopyEdit  
CREATE VIEW Coordinator_MentorStudents AS  
SELECT  
    IM.Internal_Mentor_ID,  
    MU.Name AS MentorName,  
    COUNT(S.StudentID) AS StudentCount  
FROM InternalMentor IM  
JOIN Mentor M ON IM.Internal_Mentor_ID = M.MentorID  
JOIN User MU ON M.MentorID = MU.UserID  
LEFT JOIN Student S ON IM.Internal_Mentor_ID = S.Internal_Mentor_ID  
GROUP BY IM.Internal_Mentor_ID, MU.Name;
```

General_Students:

```
sql  
CopyEdit  
CREATE VIEW General_Students AS  
SELECT  
    U.Name AS StudentName,  
    S.Field_Of_Study,  
    S.Academic_Level,  
    S.Internship_Status  
FROM Student S  
JOIN User U ON S.StudentID = U.UserID;
```

Stored Procedure

AssignStudentToMentor:

```
sql
CopyEdit
DELIMITER //

CREATE PROCEDURE AssignStudentToMentor (
    IN student_id INT,
    IN mentor_id INT
)
BEGIN
    UPDATE Student
    SET Internal_Mentor_ID = mentor_id,
        Internship_Status = 'InProgress'
    WHERE StudentID = student_id;

    SELECT
        S.StudentID, U.Name AS StudentName, S.Internship_Status,
        S.Internal_Mentor_ID
    FROM Student S
    JOIN User U ON S.StudentID = U.UserID
    WHERE S.StudentID = student_id;

END //

DELIMITER ;
```

Usage Example:

```
sql
CopyEdit
CALL AssignStudentToMentor(2, 3);
```